

The Agent Skills Guide

Five skills. Every failure mode developers hit when building with AI agents. Solved.

by JavaScript Mastery

/architect	/remember	/review	/recover	/imprint
Plan before you build	Never lose context	Catch issues early	Fix what breaks	Keep UI consistent

These five skills were built from real production builds, real failures, real sessions where the agent drifted and we had to figure out why. Each one came from a specific problem that kept repeating until we refined the solution. Install once, use on every project after today.

The System in 60 Seconds

Most developers open an AI chat, describe what they want, and react to whatever comes out. That workflow has a ceiling, and most hit it fast.

The problem is the agent has no idea what you're building. It knows what you said in the last message. Maybe a few back. But it doesn't know the decisions you made three days ago, your conventions, your folder structure, or where the project stands today. So it guesses. Guesses compound. Three weeks in, your codebase is full of contradictions nobody can fully explain.

These five skills fix that. Here's the complete workflow:

START PROJECT	<code>/architect</code>	Before any complex feature. Plan before you build.
END OF SESSION	<code>/remember save</code>	Compress what happened into a memory file
NEW SESSION	<code>/remember restore</code>	Load the memory. Pick up exactly where you left off.
AFTER UI	<code>/imprint</code>	Capture new component patterns into ui-registry
AFTER FEATURE	<code>/review</code>	Check implementation against plan before moving on
SOMETHING BROKE	<code>/recover</code>	Diagnose the failure mode and fix it in one prompt

■ Context files give the agent complete knowledge of your system. Skills make sure every session is focused, every feature is reviewed, and nothing gets lost between conversations.

/architect

Think before you build. Every time.

WHAT IT DOES

/architect reads your context files, asks focused questions one at a time, surfaces the decisions you haven't made yet, and produces a plan. It's the conversation you should be having before every serious feature, and now it happens automatically.

WHEN TO RUN IT

Before any feature that involves complex logic, multiple systems talking to each other, or decisions that are hard to reverse. If you're unsure, run it. The two minutes it takes is nothing compared to an afternoon of refactoring.

WHAT HAPPENS IF YOU SKIP IT

■ You start typing a prompt. Agent builds. Halfway through you realize you never decided how auth would work for this. Agent guessed. You're now refactoring. Skipping /architect is the single biggest source of wasted sessions.

EXAMPLE PROMPT

```
/architect feature 12
```

```
Check @context/library-docs.md for the Browserbase patterns, then /architect  
feature 13.
```

PRO TIP

■ After /architect produces the plan, review it before hitting implement. Add any decisions the agent missed. Correct any assumptions that are wrong. The plan is a starting point and you're still in control.

/remember

Never lose context between sessions.

WHAT IT DOES

/remember has two modes. **/remember save** compresses what happened in the current session, including decisions made, patterns established, and progress completed, into a memory file. **/remember restore** loads that file at the start of a new session so the agent picks up exactly where you left off.

WHEN TO RUN IT

<code>/remember save</code>	Run at the end of every session where real work happened. Especially important when a feature spans multiple sessions.	<code>/remember restore</code>	Run at the start of any session that continues work from a previous one. Replaces the need to re-explain what was built.
-----------------------------	--	--------------------------------	--

WHAT HAPPENS IF YOU SKIP IT

■ Every new session starts from zero. You spend ten minutes re-explaining what was decided yesterday. Or you don't, and the agent contradicts itself by lunch.

EXAMPLE PROMPTS

```
/remember save
```

```
/remember restore
```

THE RULE

■ `/remember restore` replaces the "read AGENTS.md" prompt for sessions where you saved memory. Don't run both, it's redundant. New feature with no previous save: read AGENTS.md. Continuing after a save: use `/remember restore`.

/review

Check the work before it becomes a problem.

WHAT IT DOES

/review checks if the implementation matched the plan, if it respects your architecture boundaries, and if it's production ready. It returns issues by severity: critical, important, and minor. It never auto-fixes. You stay in control.

WHEN TO RUN IT

After any feature that touches multiple systems, writes to the database, handles auth, or has logic that's easy to get subtly wrong. Also run it when something feels off but you can't pinpoint why.

WHAT HAPPENS IF YOU SKIP IT

■ Agent finishes a feature. You skim the code. You commit. Three days later you find the bug, in production, traced back to an assumption the agent made that you didn't catch.

EXAMPLE PROMPTS

```
/review
```

```
/review The High Match filter is showing all jobs instead of filtering by score above 70.
```

```
/review There's no download button for the generated resume. Add one next to the Extract button, visible only when the user has an existing resume.
```

PRO TIP

■ /review works best when you give it context. A plain /review is good. A /review with a specific observation about what feels wrong is better. The more specific you are, the more targeted the diagnosis.

/recover

When things break, diagnose first, fix fast.

WHAT IT DOES

/recover diagnoses which failure mode you're in and gives you the right corrective response. There are three common failure modes: a targeted bug, a polluted context, or a wrong assumption from earlier. /recover figures out which one it is and fixes it. One prompt instead of an afternoon of spiraling.

WHEN TO RUN IT

When something is completely broken and /review didn't fix it. When you've tried one corrective prompt and it didn't work. When you have a specific terminal error you need diagnosed. AGENTS.md rule: if the same problem persists after one corrective prompt, stop immediately and run /recover.

WHAT HAPPENS IF YOU SKIP IT

■ You spiral. Four hours into a session, increasingly desperate prompts, agent stuck in a loop. /recover is the circuit breaker. Run it before you waste an afternoon.

EXAMPLE PROMPTS

```
/recover [paste exact terminal error here]
```

```
/recover Feature 13 is not persisting data. After saving, the DB still shows the field as null. POST /api/agent/research is either not being called or failing silently.
```

THE GOLDEN RULE

■ Paste raw terminal logs directly into /recover, don't paraphrase. The exact error message gives the agent everything it needs to diagnose the root cause in one shot. Paraphrasing loses the detail that matters.

/imprint

Keep your UI consistent across every session.

WHAT IT DOES

/imprint captures the new component's pattern into your UI registry. Run it across the codebase and it finds inconsistencies and produces a fix list. Your design system stays coherent across every session, without you having to explain it every time.

WHEN TO RUN IT

After building any new UI component or page. Especially important when starting a new page that needs to match patterns established in previous sessions. If you're ever unsure whether a new component matches the existing design system, run /imprint.

WHAT HAPPENS IF YOU SKIP IT

■ Three sessions in, your buttons all look slightly different. Spacing is off. Your design system has fractured and you didn't notice. By the time you catch it, you're fixing twenty components instead of one.

EXAMPLE PROMPT

```
/imprint
```

HOW IT CONNECTS TO CONTEXT FILES

/imprint writes to **ui-registry.md** automatically. This file starts empty and fills up as you build. By page three of your project, the agent knows exactly how your cards, buttons, and inputs look, and matches them without being told. This is especially powerful when building without a Figma design. The ui-registry becomes your design system.

■ You don't need to run /imprint manually if AGENTS.md already tells the agent to update ui-registry.md after every feature. Check if it auto-ran first — if ui-registry.md updated, you're already done.

Quick Reference

SKILL	WHEN TO RUN	PROMPT TO USE	SKIP COST
/architect	Before any complex feature with real logic or multiple systems	<code>/architect feature [N]</code>	Wrong assumptions → hours of refactoring
/remember save	End of every session where real work happened	<code>/remember save</code>	Next session starts from zero, contradictions pile up
/remember restore	Start of any session continuing from a previous save	<code>/remember restore</code>	10 minutes re-explaining context that was already saved
/imprint	After building any new UI component or page	<code>/imprint</code>	Design system fractures silently across sessions
/review	After any logic-heavy feature before moving on	<code>/review /review [specific observation]</code>	Bugs slip through, caught in production not development
/recover	When broken after one corrective prompt didn't work	<code>/recover [paste terminal error]</code>	Hours of spiraling prompts, agent stuck in a loop

Install the skills: `npx skills@latest add JavaScript-Mastery-Pro/skills`